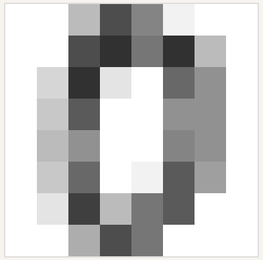
Representation learning & autoencoders

From handcrafted features to learned representations.

Anush Sankaran

The pipeline we already know.



Observed data

→

Features

Chosen measurements

→

Neural network

Trainable weights

→

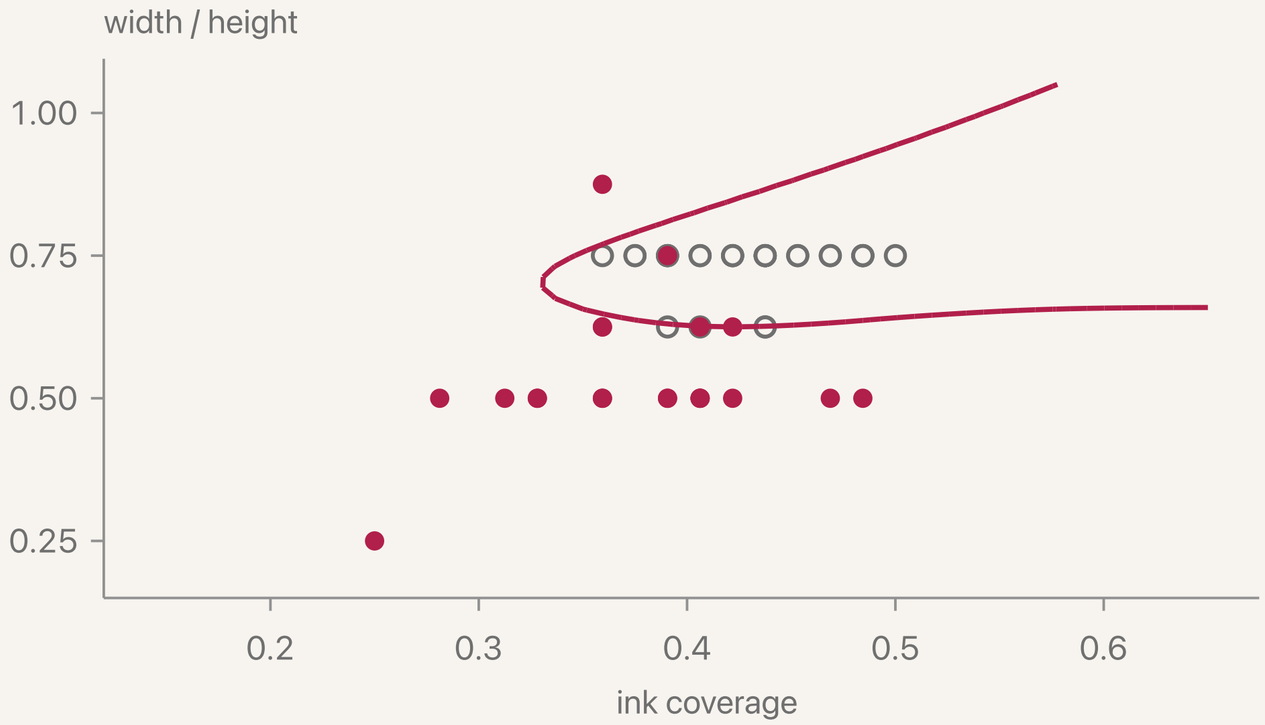
Class

Prediction

Training examples + class labels → learn network weights

Data → features → learned decision rule.

The network learns a boundary in feature space.



digit 0 digit 1

2 → 8 → 1

Two features.
One classifier.

The boundary is learned.
The coordinates were chosen.

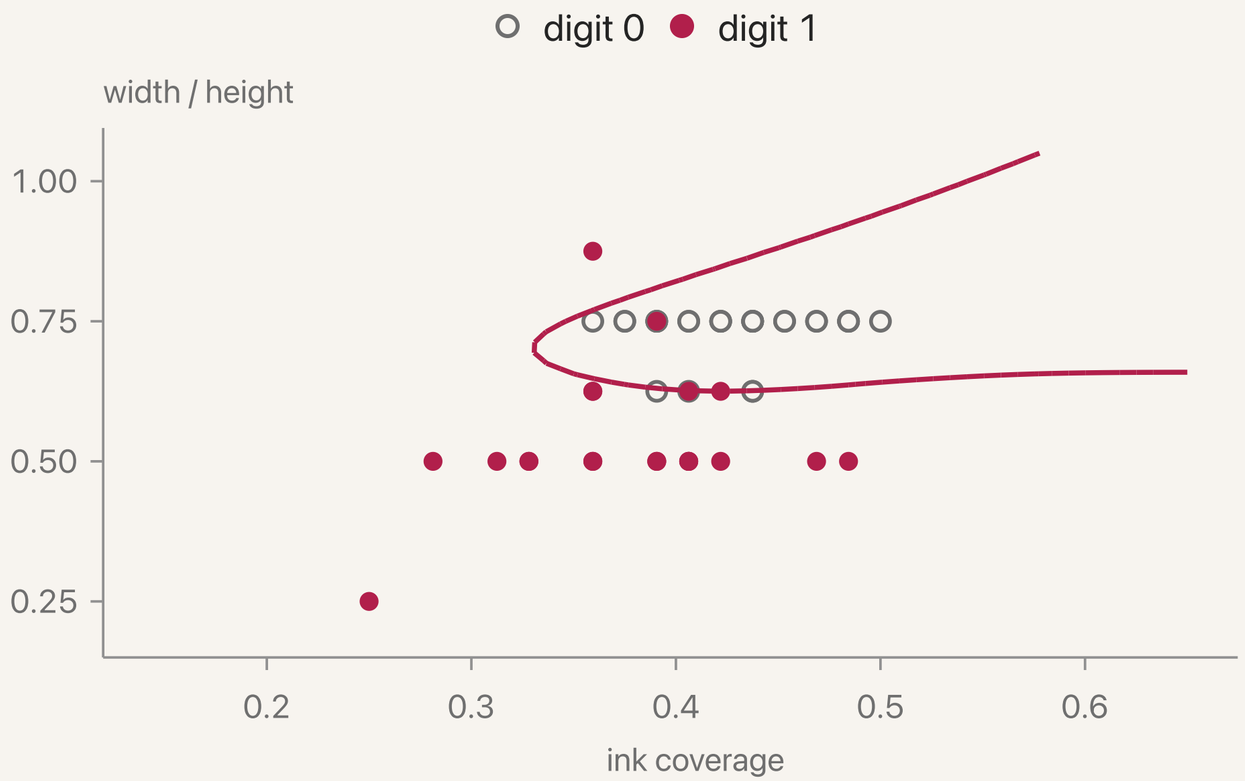
Real digit data; trained two-feature model. Demonstration, not a benchmark claim.

Who designed those coordinates?

We did.

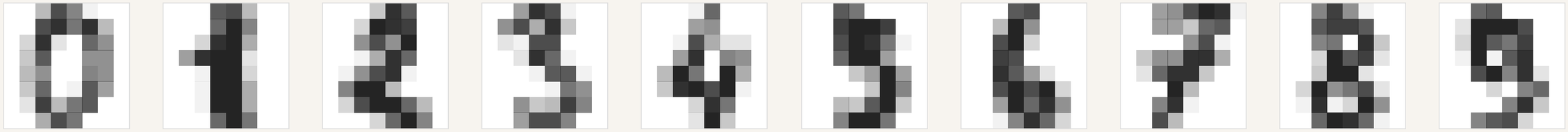
$$z = \phi(x)$$

We chose what to measure.
The network learned what to do with those measurements.



From handcrafted to learned features

One familiar problem: reading handwritten digits.



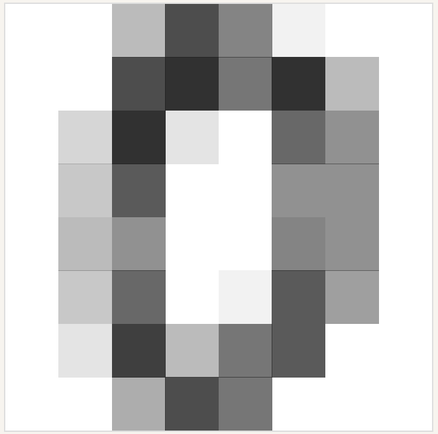
0 1 2 3 4 5 6 7 8 9

Different handwriting.
Same digit identity.

8 × 8 → 64 values

Real UCI optical digits / 1,797 available examples / pixel values 0–16.

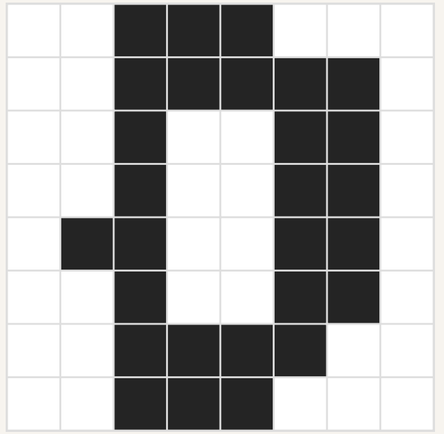
First, define what counts as ink.



Available pixel values

$$B_{rc} = 1[I_{rc} \geq 5]$$

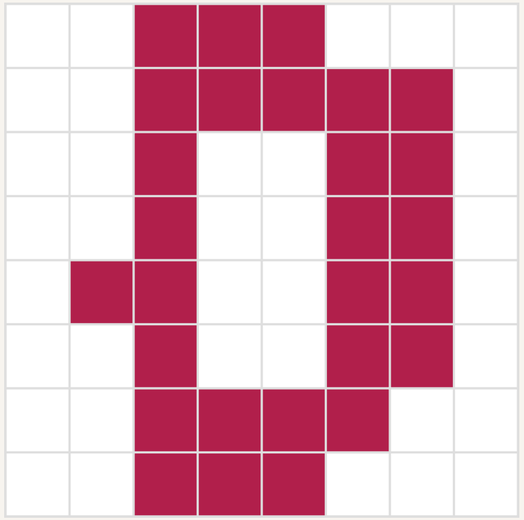
Our threshold.
Our preprocessing choice.



Binary foreground mask

Follow this same digit through six transformations.

How much of the image contains ink?



Count foreground cells

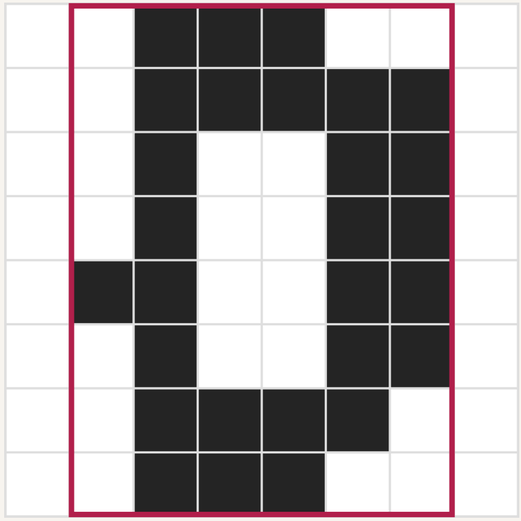
$$f_1 = \Sigma_{r,c} B_{rc} / 64$$

$$28 / 64$$

$$f_1 = 0.438$$

UCI optical digits / sample 0 / foreground: pixel value ≥ 5.

How wide is the writing relative to its height?



Smallest foreground bounding box

f_2 = width / height

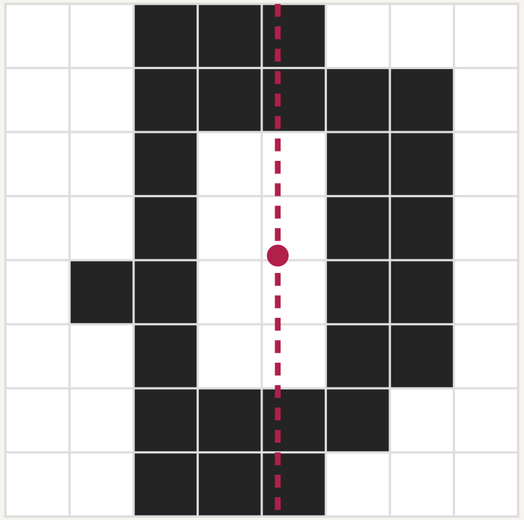
6 columns

8 rows

$f_2 = 0.750$

UCI optical digits / sample 0 / foreground: pixel value ≥ 5.

Where is the horizontal center of ink?



Mean foreground column

$$f_3 = \Sigma_{r,c} c B_{rc} / (7 \Sigma_{r,c} B_{rc})$$

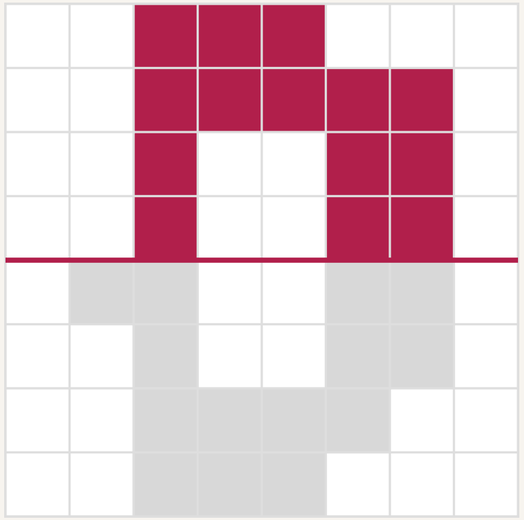
$c \in \{0, 1, \ldots, 7\}$

0.536

normalized horizontal center

UCI optical digits / sample 0 / foreground: pixel value ≥ 5.

Is more ink above or below the middle?



Top four rows highlighted

f_4 = upper ink / total ink

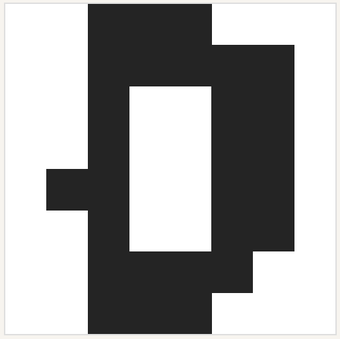
14 upper cells

28 all ink cells

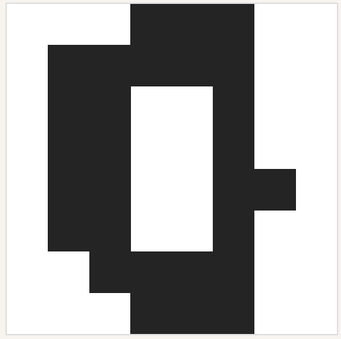
$f_4 = 0.500$

UCI optical digits / sample 0 / foreground: pixel value ≥ 5.

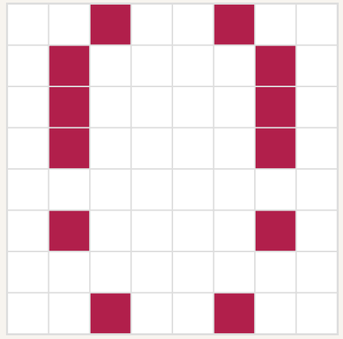
How similar is the digit to its mirror?



Original mask



Left-right reflection



Mismatched cells

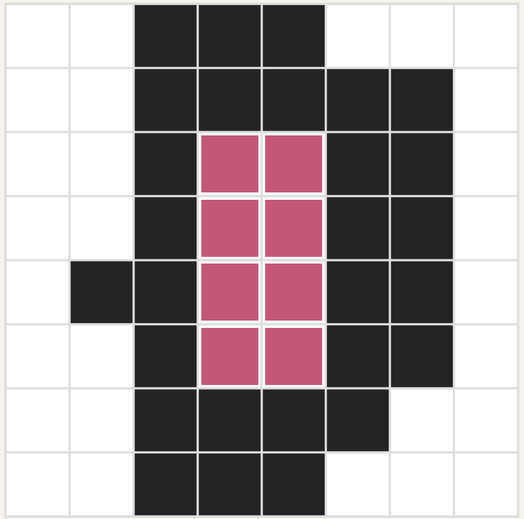
0.813

symmetry score

$$f_5 = 1 - \Sigma_{r,c} |B_{rc} - B_{r,7-c}| / 64$$

UCI optical digits / sample 0 / foreground: pixel value ≥ 5.

Does the writing enclose an empty region?



Enclosed background highlighted

f_6 = number of holes

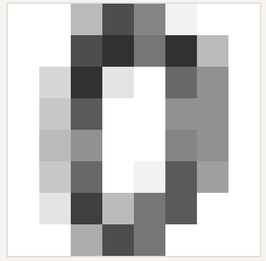
1

enclosed region

Trace background regions.
Exclude those touching the image border.

UCI optical digits / sample 0 / foreground: pixel value ≥ 5.

One image becomes six named numbers.



Same original zero

→

Ink	Aspect	Center	Upper	Symmetry	Holes
0.438	0.750	0.536	0.500	0.813	1

$$z = \phi(x) \in \mathbb{R}^6$$

Each coordinate has a human-designed definition.

From handcrafted to learned features

The extractor is fixed. The classifier is trained.



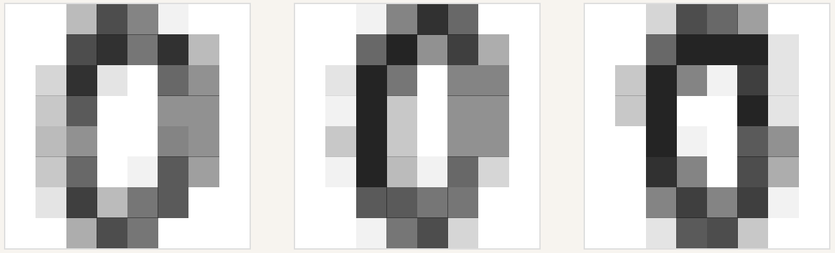
Hand-engineered preprocessing

Trainable neural classifier

Training the downstream network does not update these six extraction rules.

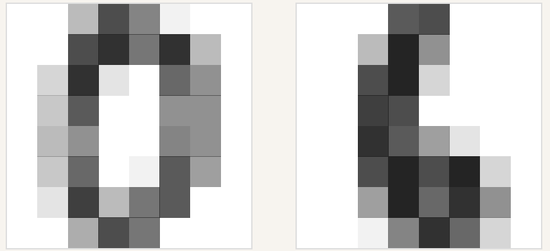
A new style or task can expose what we discarded.

Same digit, different writing



Thickness, alignment, and gaps can change our measurements.

Different question



Recognizing 0 versus 6 may need details that a hole count misses.

New conditions may require new measurements, normalization, or combinations.

Could the network learn the feature extractor?

Pixel array
64 available values

→

Learned features
$z = f_\theta(x)$

→

What target?
what teaches f_θ?

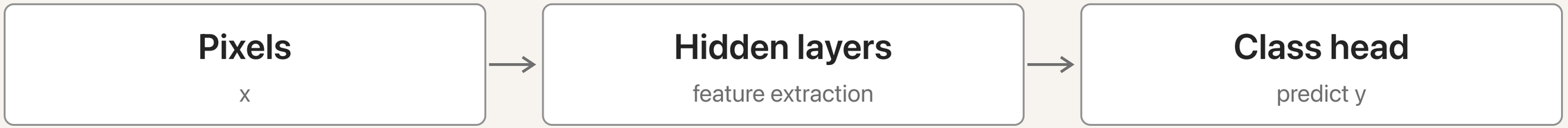
Class label
Supervised feature learning

The input itself
Reconstruction-based learning

Keep basic formatting and scaling. Stop specifying the six descriptors.

Replace the class head with a reconstruction path.

CLASSIFICATION



RECONSTRUCTION

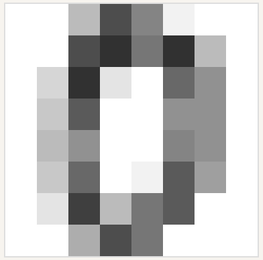


Same neural-network building blocks. A different output and objective.

From handcrafted to learned features

The target is the input, not the digit label.

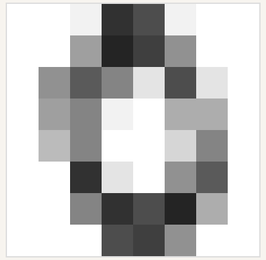
Classifier



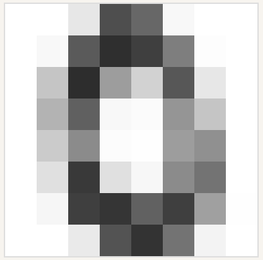
→ 0

$L_{class}(y, \hat{y})$

Autoencoder



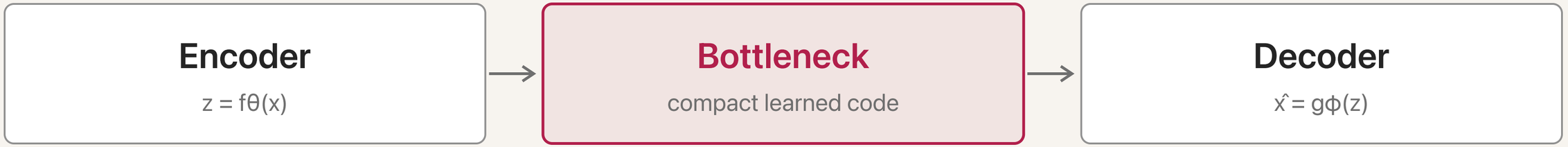
→



$L_{rec}(x, \hat{x})$

Reconstruction supplies a training signal without class annotations.

Reconstruction error teaches the feature map.



$$\min_{\theta,\phi} \; (1/64) \sum_{j=1}^{64} (x_j - \hat{x}_j)^2$$

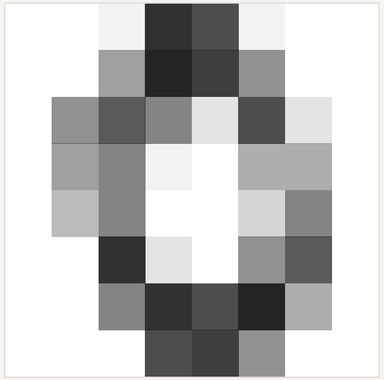
Backpropagation updates both networks.

A bottleneck or another constraint discourages a trivial copy.

The code is learned; the reconstruction is visible.

Digit 0 | Digit 1 | Digit 6 | Held-out examples

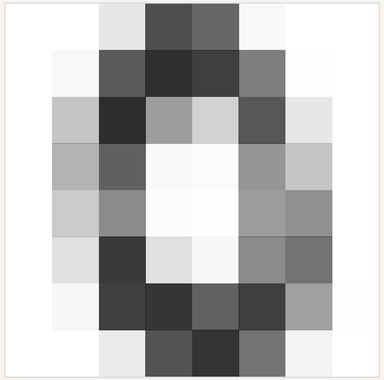
INPUT X



LEARNED CODE Z

z_1	z_2	z_3	z_4
-0.37	0.43	0.52	0.44
z_5	z_6	z_7	z_8
0.37	-0.23	0.49	-0.40

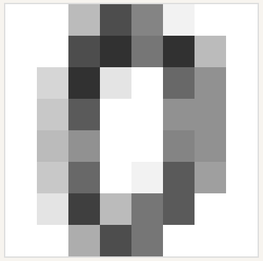
RECONSTRUCTION



Test sample 49 / eight latent values / no class-label loss

Measured teaching demonstration / 64 → 32 → 8 → 32 → 64.

Keep the encoder. Add a head for the task.



AVAILABLE PIXELS

Encoder

learned from reconstruction

Digit identity

a new supervised head

Odd or even

another supervised head

The encoder can be reused; each supervised head still needs its own training signal.

Named measurements and learned codes have different strengths.

Handcrafted

[ink, aspect, center, ...]

Named meanings.
Auditable transformations.
Explicit assumptions.

Learned

[z_1, z_2, z_3, ...]

Data-adapted combinations.
Often distributed information.
Harder to name each coordinate.

Meaning can be spread across coordinates instead of assigned to one coordinate.

Learning without labels opens the door to reuse.

Designed for known signals

Handcrafted features may need redesign.

Learned from input structure

Autoencoder features may support several tasks.

Still shaped by the training data

Transfer is a hypothesis to test.

Label-independent does not mean universally useful.

Compact representations are not a new idea.

PCA

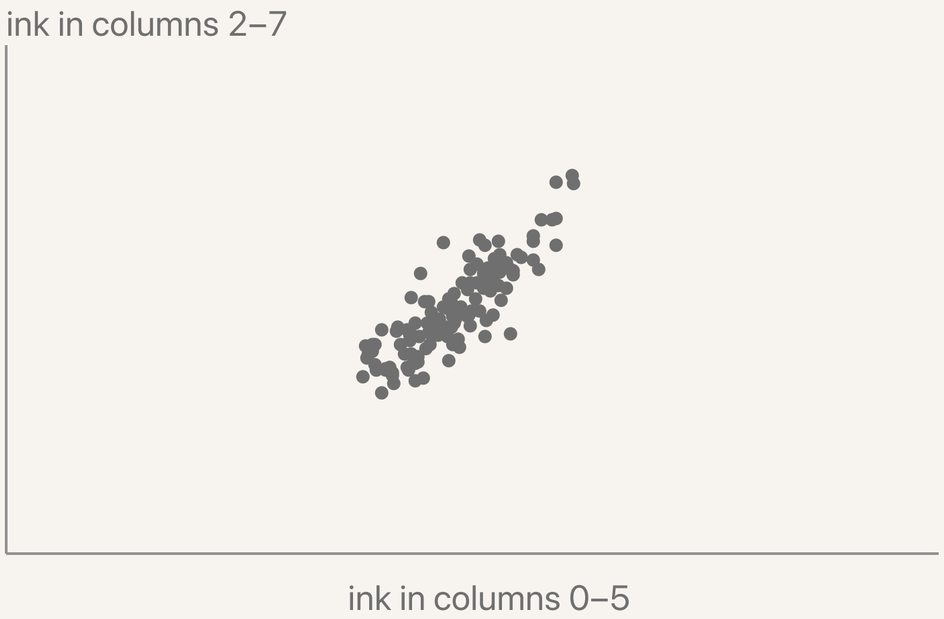
Principal component analysis

64 values → k coordinates

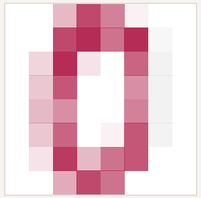
Learn a compact linear description
of how the data varies.

PCA is representation learning with a linear constraint.

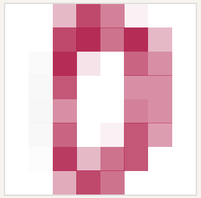
Two measurements can tell almost the same story.



Each dot is one digit.



columns 0–5



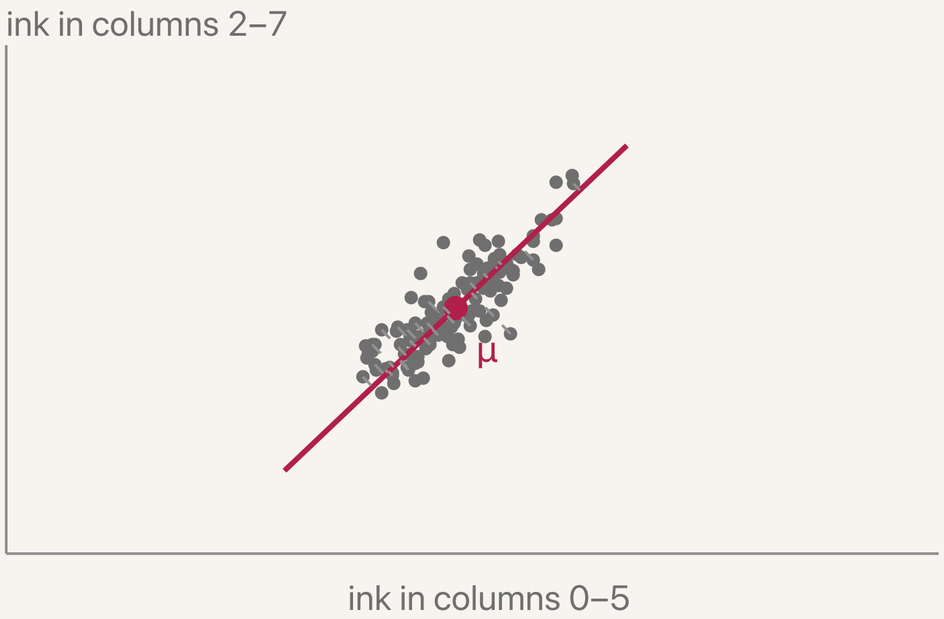
columns 2–7

Measure ink in two overlapping windows.

Their ink amounts often rise together.

PCA looks for a less redundant coordinate system.

Place a ruler along the cloud's longest direction.



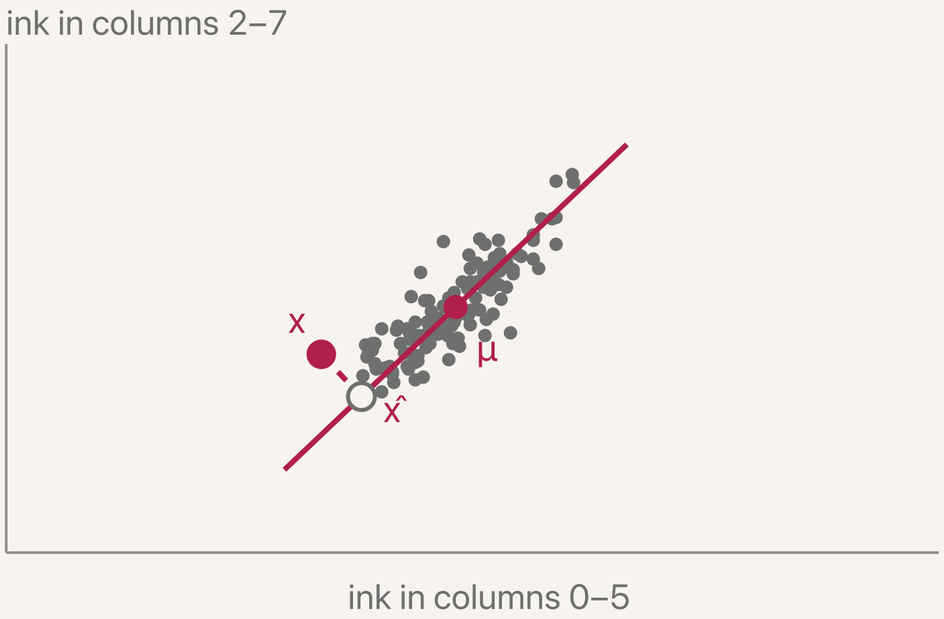
Center at the mean.

Turn the ruler to capture the largest spread.

Drop each point onto the ruler. Keep its signed position.

One coordinate retains 92.3% of variance in these two measurements.

One number locates an approximation of the point.



$z = v_1^T(x - \mu)$

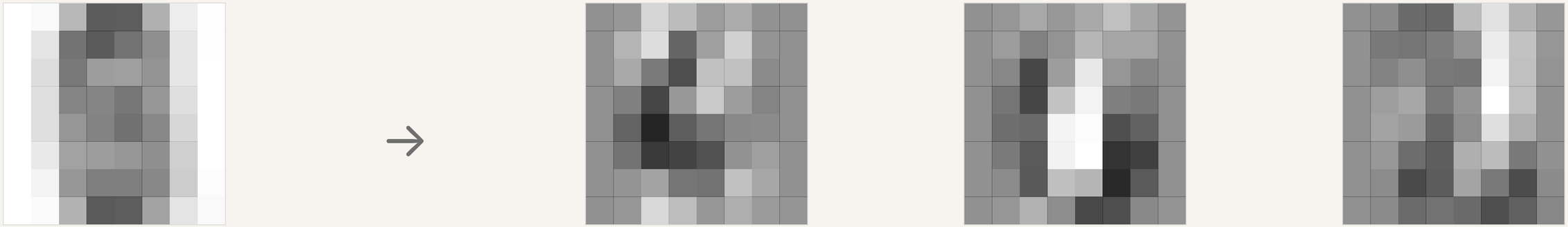
Encode: how far along the ruler?

$\hat{x} = \mu + zv_1$

Decode: walk that far from the mean.

The code preserves position along the line. It discards the off-line residual.

PCA keeps directions of greatest variance.



Training mean | Direction 1 | Direction 2 | Direction 3

$$C\,v_j = \lambda_j v_j \quad \lambda_1 \geq \lambda_2 \geq \ldots$$

An eigenvector is a direction. Its eigenvalue measures the spread along it.

SVD organizes the same information.

X_c = U × Σ × V^T



CENTERED DATA

Rows: digits
Columns: pixels

EXAMPLE COORDINATES

Normalized positions
along each direction

DIRECTION STRENGTHS

Singular values
$\sigma_1 \geq \sigma_2 \geq \ldots$

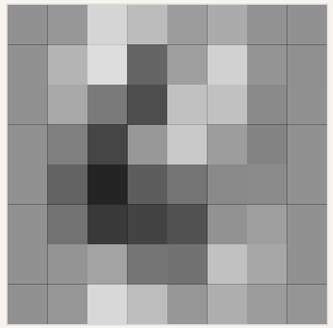
PIXEL DIRECTIONS

The rulers
PCA uses

Actual PCA scores = UΣ. SVD can compute PCA without forming a covariance matrix.

Keep the strongest terms to get a compact code.

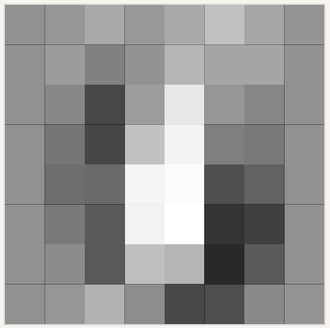
SHARED DIRECTION 1



$\sigma_1 u_1 v_1^T$

+

SHARED DIRECTION 2



$\sigma_2 u_2 v_2^T$

+ …

$$\hat{X}_c = U_k \Sigma_k V_k^T$$

Keep k directions.
Keep k scores per example.

$$\lambda_j = \sigma_j^2 / (n - 1)$$

PCA is the representation choice. SVD is a direct way to compute its directions and scores.

Project into the code. Reconstruct from the code.

Center

$$x_c = x - \mu$$

Encode

$$z = V_k^T x_c$$

Decode

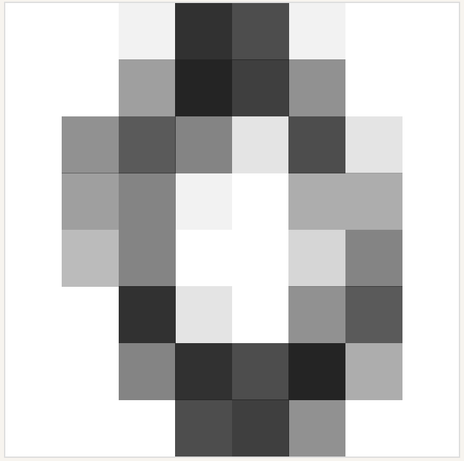
$$\hat{x} = \mu + V_k z$$

$$V_k = [v_1, \ldots, v_k] \quad V_k^T V_k = I$$

A lower-dimensional code generally loses information.

More coordinates preserve more image detail.

2 components | 8 components | 16 components | 32 components



Held-out input

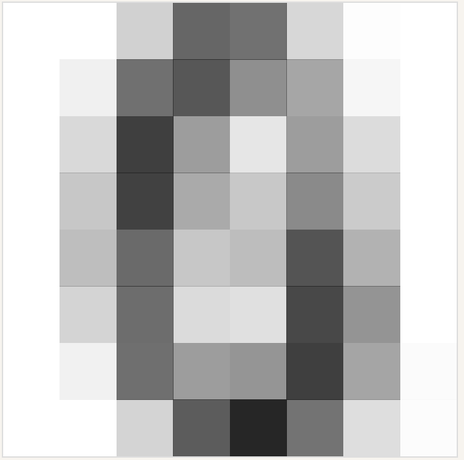
→

2

COORDINATES

→

PCA RECONSTRUCTION



28.4% of training variance retained

Same input. Same fitted PCA. Only k changes.

From handcrafted to learned features

Autoencoders can go beyond a linear subspace.

PCA

$z = V_k^T(x-\mu)$

Linear directions.
Closed-form decomposition.

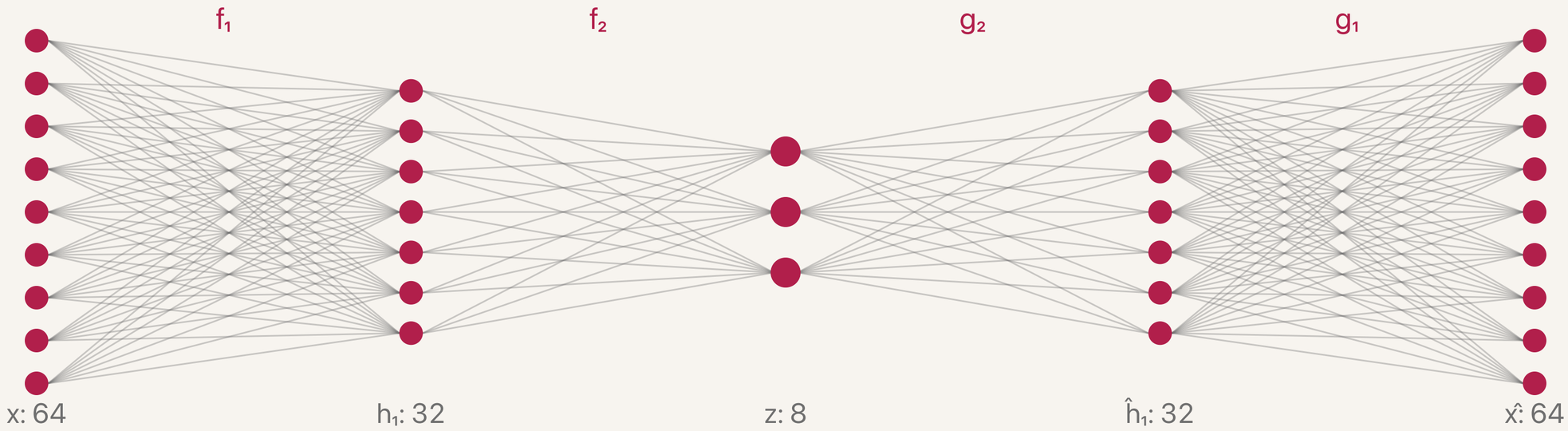
Nonlinear autoencoder

$z = f_\theta(x)$

Learned nonlinear maps.
Optimization and capacity choices.

Linear AE + squared error connects to PCA; nonlinear layers extend the model family.

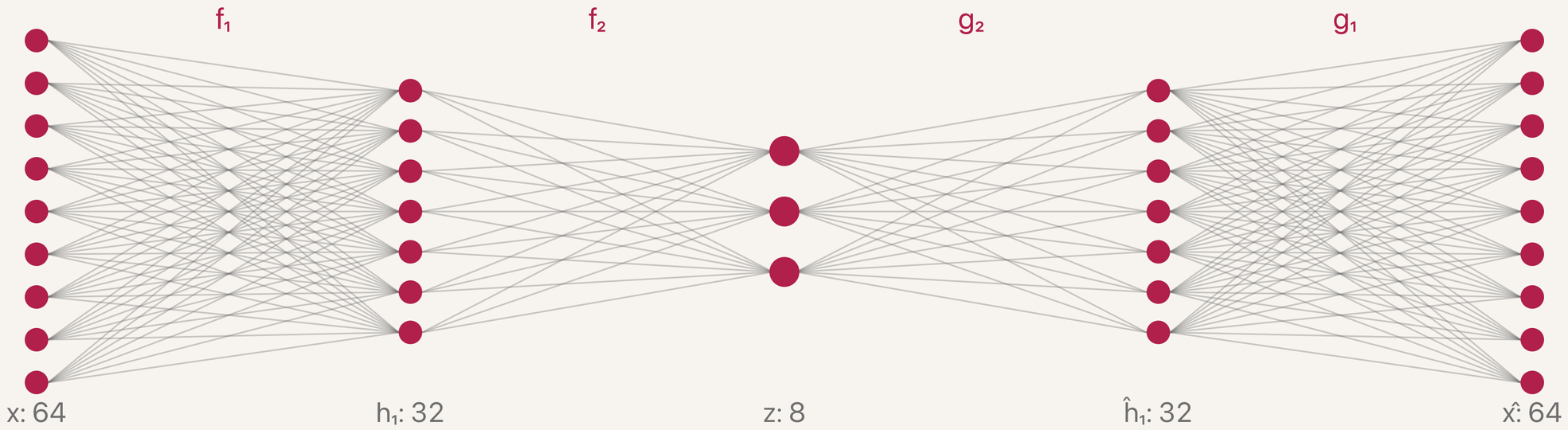
Stacking gives the encoder several transformations.



$$x \rightarrow f_1 \rightarrow f_2 \rightarrow z \rightarrow g_2 \rightarrow g_1 \rightarrow \hat{x}$$

64 → 32 → 8 → 32 → 64 / schematic dots; exact dimension labels.

Layerwise pretraining learns one encoder at a time.

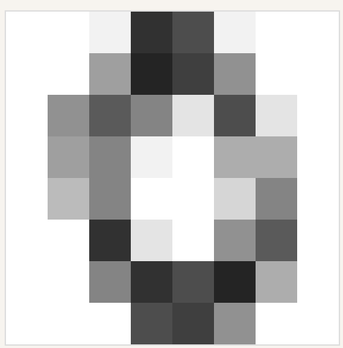


3. Compose the encoders and decoders; optionally fine-tune jointly.

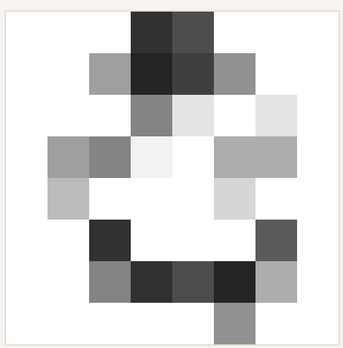
1 Learn f_1 2 Learn f_2 3 Assemble / fine-tune

Stack the learned encoders; optionally fine-tune the full network.

Denoising changes the input, not the clean target.



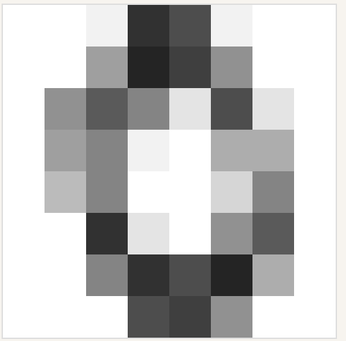
Clean training example



Corrupted input

Encoder + decoder

learn to recover



CLEAN TARGET

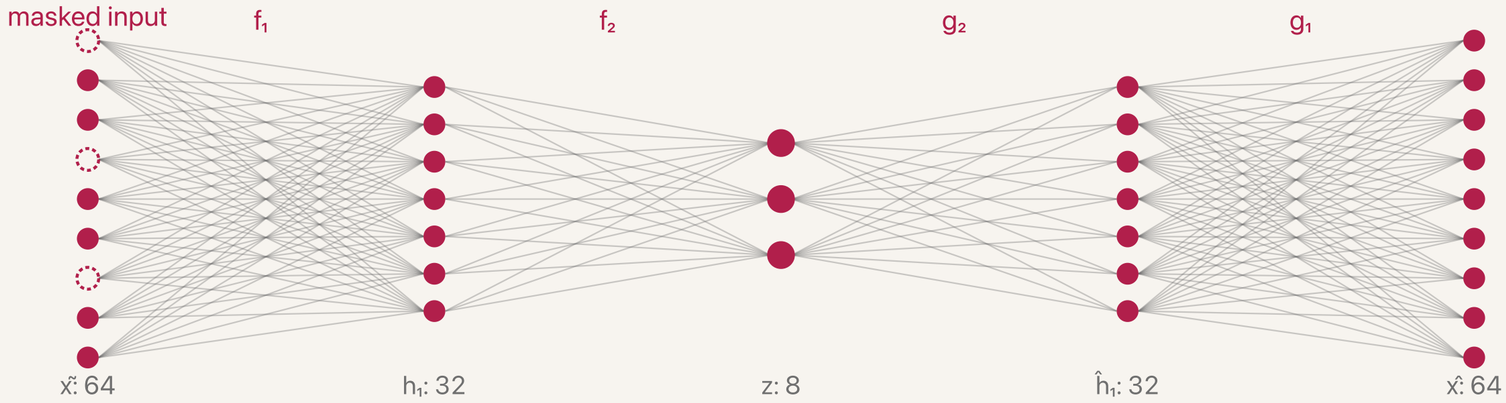
$$\min \; E_{\tilde{x} \sim q(\cdot|x)} \, \|x - g_\phi(f_\theta(\tilde{x}))\|^2$$

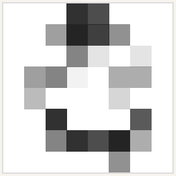
Learning to recover missing evidence can discourage trivial copying.

Recovery is learned, and it remains imperfect.

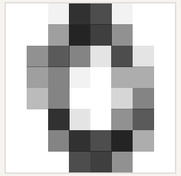
Digit 0 | Digit 1 | Digit 6

Test sample 49 / 25% masking

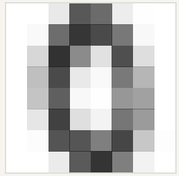




Corrupted $\tilde{x}$
enters encoder



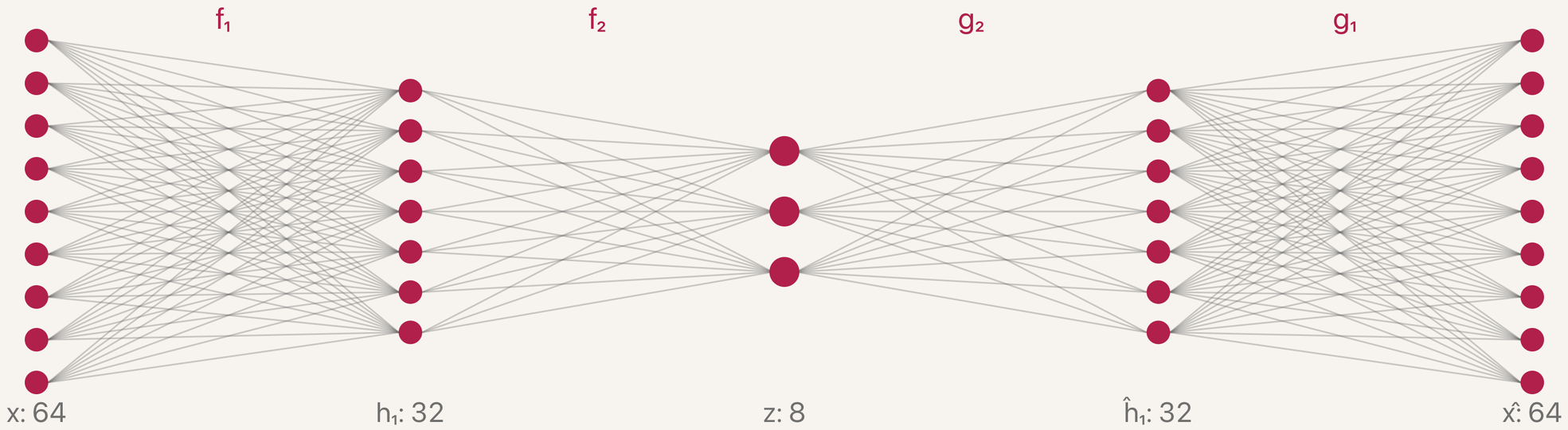
Clean x
loss target



Decoded $\hat{x}$
compare with x

Same network / corrupted input → learned code → output / loss compares output with clean x.

Stack denoising tasks at successive layers.



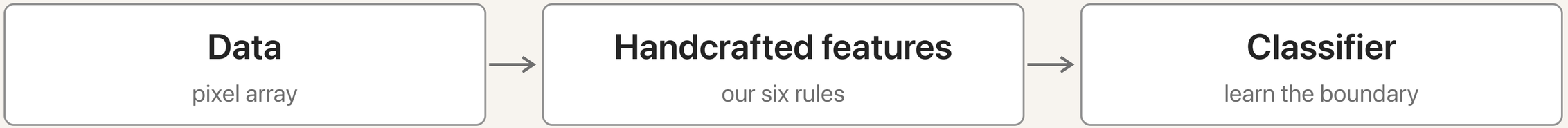
3. Stack f_1 and f_2; optionally fine-tune jointly.

1 Learn f_1 2 Learn f_2 3 Assemble / fine-tune

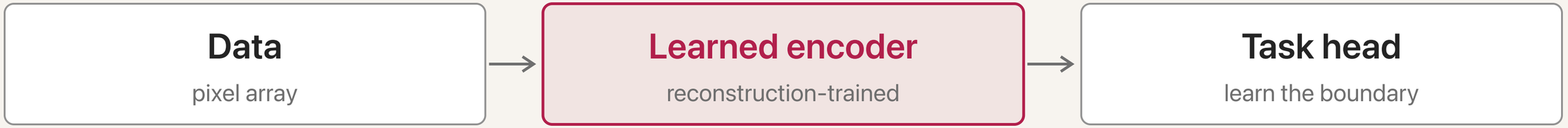
Denoising supplies the local objective. Stacking supplies the hierarchy.

Return to the pipeline—with a learned extractor.

BEFORE

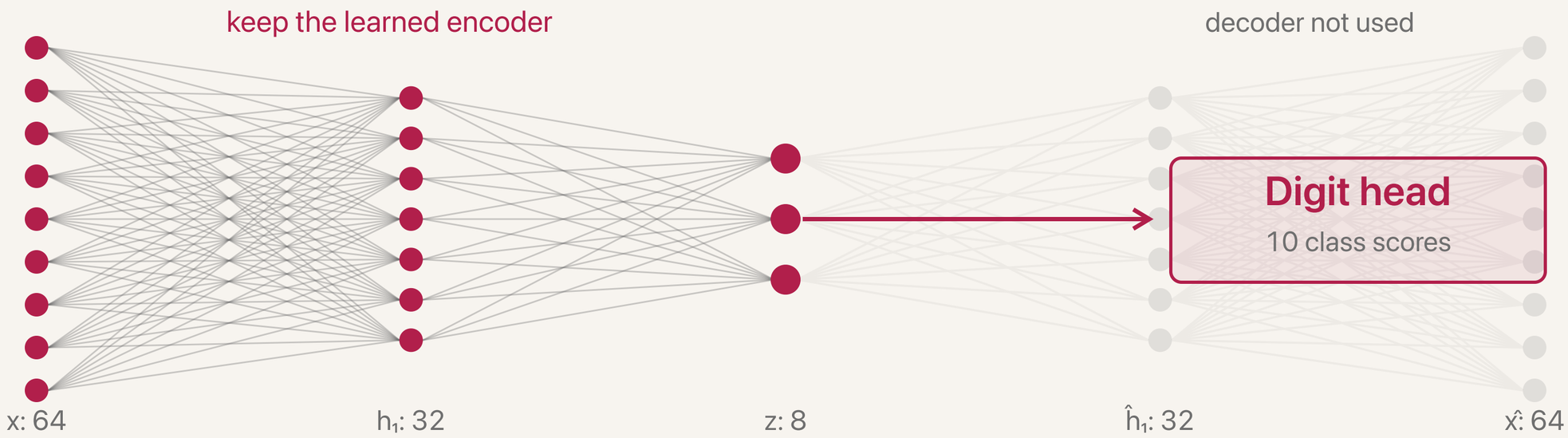


AFTER REPRESENTATION PRETRAINING



We did not stop learning decision boundaries. We also learned the coordinates.

Keep the encoder; replace the decoder with a head.



Classification: train a ten-class head on the same learned code.

Retain f_1 and f_2

Remove g_2 and g_1

Train head; optionally fine-tune encoder

Reconstruction learned the representation. Labels now teach the decision boundary.

What did we learn: a boundary, features, or both?

Handcrafted pipeline

We design the features.
Labels train the boundary.

Autoencoder pretraining

Reconstruction trains
the feature map.

Downstream fine-tuning

Labels can adapt
both features and boundary.

Representation learning asks the model to learn useful ways to describe the data.

What produced the numerical examples?

Data

UCI optical digits, 1,797 rows.
Split: 1,252 / 270 / 275.
Stratified split seed: 42.

Autoencoders

64–32–8–32–64; tanh / sigmoid.
Adam: 0.002; batch: 64; MSE.
Initialization seed: 17.

Selection

Validation-selected checkpoints.
AE: epoch 350; DAE: 343.
Denoising: 25% masking.

Examples

Test IDs: 49 (0), 11 (1), 26 (6).
First test instance of each class.
No stacked-pretraining result claimed.

These small local runs illustrate mechanisms; they are not state-of-the-art claims.

Sources and data provenance.

UCI: Optical Recognition of Handwritten Digits

Real eight-by-eight observations; values 0–16.

Used through the public scikit-learn digits distribution.

scikit-learn: PCA

Centering, principal axes, and projection.

Numerical PCA here was fitted on the training split only.

Goodfellow, Bengio & Courville, 2016

Deep Learning, Chapters 14–15.

Autoencoders and representation learning.

Vincent et al., JMLR 2010

Stacked Denoising Autoencoders.

Layerwise learning from local denoising objectives.

Observed inputs, computed transformations, and measured model outputs are distinguished throughout.